

stoichiometric_justification

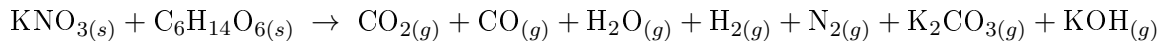
June 8, 2026

1 Stoichiometric Analysis of KNSB Composite Sugar Propellant

1.1 Background

KNSB (Potassium Nitrate / Sorbitol) is a castable solid rocket propellant whose oxidiser–fuel ratio must be carefully chosen to balance performance, safety, and manufacturability. To determine that ratio rigorously we need a **balanced combustion equation**.

The reaction under study is:



We will 1. implement a **Python translation of the Mazza nullspace algorithm** (Mazza & Canuto, *Fundamental Chemistry with MATLAB*, Elsevier 2022, Chapter 2), and 2. use it to balance a thermochemically consistent, simplified form of the equation, 3. derive the ideal stoichiometric mass ratio, and 4. justify why a slightly fuel-rich blend is preferred in practice.

Ferric oxide (FeO) acts as a catalyst and does not appear in the equation.

1.2 1. Python port of the Mazza nullspace method

1.2.1 1.1 Theory (Chapter 2, Mazza & Canuto)

Given n substances (reactants and products) containing m elements, label the signed atom-count matrix $\mathbf{A} \in \mathbb{Z}^{m \times n}$:

$$A_{jk} = \begin{cases} +a_{jk} & \text{substance } k \text{ is a reactant} \\ -a_{jk} & \text{substance } k \text{ is a product} \end{cases}$$

where a_{jk} is the number of atoms of element j in substance k . Atom conservation then reads

$$\mathbf{A} \mathbf{x} = \mathbf{0}, \quad \mathbf{x} = [x_1, \dots, x_n]^\top \in \mathbb{Z}_{>0}^n$$

The solution lives in the **nullspace** of $\mathbf{A}$.

A unique (up to scale) positive integer solution exists when $\dim(\ker \mathbf{A}) = 1$, i.e. the nullspace is one-dimensional. This is the *minimum-balance stoichiometry*.

Mazza's algorithm (adapted from MATLAB to Python/SymPy): 1. Parse each chemical formula as a symbolic expression. 2. Build **A** by reading atom counts from the symbolic tree. 3. Compute $\ker \mathbf{A}$ exactly with `Matrix.nullspace()`. 4. Convert the rational nullspace vector to the minimum positive-integer form.

1.2.2 1.2 Environment

```
[1]: from sympy import symbols, Symbol, Mul, Pow, Matrix, Rational, lcm as sym_lcm
from math import gcd
from functools import reduce
from fractions import Fraction
```

1.2.3 1.3 Formula parser

Translates the idea behind MATLAB's `symvar` / `children` traversal: every symbolic expression is either a bare `Symbol`, a `Pow` (element with exponent), or a `Mul` (product of the previous two). We walk the expression tree recursively and collect `{element: count}` pairs.

```
[ ]: def _parse_formula(expr, elem_set: set) -> dict:
    """
    Recursively parse a SymPy symbolic expression into an atom-count dict.

    Mirrors the MATLAB `children()` traversal in Mazza (2.4.3)

    Parameters
    -----
    expr      : SymPy expression (Symbol, Pow, or Mul)
    elem_set  : set of SymPy symbols - the chemical elements in the reaction

    Returns
    -----
    dict {symbol: int} atom counts for this formula
    """
    counts: dict = {}

    if isinstance(expr, Symbol):                # e.g. O, K, H
        if expr in elem_set:
            counts[expr] = 1

    elif isinstance(expr, Pow):                 # e.g. O**3, H**14
        base, exp = expr.args
        if base in elem_set:
            counts[base] = int(exp)

    elif isinstance(expr, Mul):                 # e.g. K*N*O**3, C**6*H**14*O**6
        for factor in expr.args:
            for elem, n in _parse_formula(factor, elem_set).items():
                counts[elem] = counts.get(elem, 0) + n
```

```
return counts
```

1.2.4 1.4 stoichiometry() — the core function

Direct Python equivalent of the `stoichiometry.m` function in Mazza §2.5.1. The main structural difference from the MATLAB version: - `Matrix.nullspace()` returns exact rational vectors; no need for a separate `rat()` call. - We use `functools.reduce + math.gcd` for the LCM/GCD reduction.

```
[3]: def stoichiometry(
    elements: list,
    substances: list,
    n_reactants: int,
    label: str = "Reaction",
) -> list:
    """
    Minimum-balance stoichiometry via the Mazza nullspace algorithm.

    Port of `stoichiometry.m` (Mazza & Canuto, Chapter 2) from MATLAB to Python/
    ↪SymPy.

    Parameters
    -----
    elements      : list of SymPy symbols - chemical elements present (e.g. [K, N,
    ↪O, C, H])
    substances    : list of SymPy expressions - all substances, reactants first,
    ↪then products
                    written as symbolic products: K*N*O**3, C**6*H**14*O**6, etc.
    n_reactants   : int - number of reactant substances (prefix of `substances`)
    label         : str - human-readable reaction name for the printed header

    Returns
    -----
    int_coeffs   : list[int] - minimum positive integer stoichiometric
    ↪coefficients,
                    in the same order as `substances` (reactants then products)

    Raises
    -----
    ValueError   - if the nullspace is empty (no solution) or multi-dimensional
                    (underdetermined system; try removing linearly dependent
    ↪species)
    """
    m = len(elements)          # number of elements
    n = len(substances)        # number of substances
    elem_set = set(elements)
```

```

print(f"\n{' '*55}")
print(f"  Minimum-balance stoichiometry - {label}")
print(f"{' '*55}")
print(f"  Elements : {m}    Substances : {n}")

# ----- #
# Step 1 - build the signed balance matrix A (m x n)
# Mazza eq. (2.16): A_{jk} = +count for reactants, count for products
# ----- #
A = Matrix.zeros(m, n)

for k, formula in enumerate(substances):
    sign = 1 if k < n_reactants else -1
    atom_counts = _parse_formula(formula, elem_set)
    for j, elem in enumerate(elements):
        A[j, k] = sign * atom_counts.get(elem, 0)

# ----- #
# Step 2 - nullspace (exact, rational)
# MATLAB: Z = null(A, 'r') → SymPy: A.nullspace()
# ----- #
null_vecs = A.nullspace()
dim_null = len(null_vecs)

if dim_null == 0:
    raise ValueError(
        "Nullspace is empty - the system is inconsistent. "
        "Check that all elements and species are correctly specified."
    )
if dim_null > 1:
    print(
        f"    WARNING: nullspace dimension = {dim_null} > 1 - system is
→underdetermined.\n"
        f"    The reaction as written has linearly dependent species and
→cannot\n"
        f"    be given unique stoichiometric coefficients. Simplify the
→product set."
    )
    return None

z = null_vecs[0]    # column vector of Rational entries

# ----- #
# Step 3 - convert to minimum positive integers
# MATLAB: [nZ,dZ]=rat(Z); intZ=Z*lcm(sym(dZ))
# ----- #
fracs = [Fraction(str(val)) for val in z]    # exact fractions

```